FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

INOVALAB: SISTEMA JAVA SOULPET

Grupo:
Bianca Pereira da Silva - RM: 571077
Isabelle Souza Lima Pires Araujo - RM: 569370
Maria Eduarda Cavallari Quarelo - RM: 570462
Thays Lira de Oliveira - RM: 568799

SÃO PAULO
2026

SUMÁRIO

1. OBJETIVO E ESCOPO DO PROJETO **2**

2. DESCRIÇÃO DAS FUNCIONALIDADES DA SOLUÇÃO **2**

3. TABELA DE ENDPOINTS (API RESTFUL) **3**

4. PROTÓTIPO DO SISTEMA **5**

5. MODELO DE ENTIDADE-RELACIONAMENTO (MER) **9**

6. DIAGRAMA DE CLASSES **9**

7. PROCEDIMENTOS PARA RODAR A APLICAÇÃO **11**

7.1 Ferramentas e versões necessárias 11

7.2 Como importar o projeto 11

7.3 Configuração do banco de dados 12

7.4 Executando a aplicação localmente 12

7.5 Testando os endpoints 12

7.6 Acesso ao deploy 12

1. OBJETIVO E ESCOPO DO PROJETO

A SoulPet, proposta de solução da InovaLab para a SoulUp, não busca criar um aplicativo do zero, mas sim incrementar uma feature dentro da plataforma da própria SoulUp, funcionando como um marketplace de impacto voltado para a proteção animal. A feature conta com missões sustentáveis que os usuários realizam para acumular pontos em suas carteiras de pontuação, podendo trocá-los por itens para seu pet ou apoiar ONGs ativas na plataforma, tudo centralizado em uma única tela, de forma simples e acessível.

Nas missões da SoulPet, os usuários completam tarefas como curtir posts, compartilhar campanhas de adoção e engajar para ajudar pets abandonados a encontrar um lar. Cada ação tem uma pontuação clara e justa, e um sistema de níveis, que vai de Protetor Iniciante (menor nível) a Herói Animal (maior nível), mantém o usuário cada vez mais ativo dentro da plataforma.

A Loja Solidária é o espaço onde os pontos acumulados nas missões são utilizados. Nela, o usuário pode trocar seus pontos por produtos como ração, brinquedos e acessórios, sendo que parte do valor vai diretamente para ONGs parceiras de proteção animal. O usuário também pode optar por doar seus pontos diretamente a essas ONGs.

O escopo da SoulPet se limita ao gerenciamento de usuários, missões, pontos e recompensas, além do vínculo com as comunidades/ONGs parceiras. A adoção de animais acontece através das ONGs parceiras e está fora da plataforma da SoulPet.

Nesta etapa, a solução é implementada como uma API RESTful em Java 21, organizada em camadas (TO, DAO, BO e Resource) e integrada a um banco Oracle, para ser consumida pelo front-end da plataforma.

2. DESCRIÇÃO DAS FUNCIONALIDADES DA SOLUÇÃO

- **Gestão de usuários**: cadastro, consulta, atualização e exclusão de usuários, com controle de pontos e nível.
- **Missões sustentáveis**: cadastro, consulta, atualização e exclusão de missões, cada uma com pontuação e vinculada a uma comunidade/ONG parceira.

- **Conclusão de missões**: registro da missão de cada usuário, com status de conclusão e data de conclusão.
- **Recompensas**: cadastro, consulta, atualização e exclusão de recompensas (produtos ou doações), com custo em pontos e controle de estoque.
- **Resgate de recompensas**: registro dos resgates feitos pelos usuários, com a data do resgate.
- **Comunidades/ONGs parceiras**: cadastro, consulta, atualização e exclusão das ONGs que recebem o engajamento e as doações.
- **Validações e tratamento de exceções**: validação dos dados de entrada com @Valid e tratamento de exceções com try/catch, devolvendo respostas HTTP adequadas para cada situação (201, 204, 400 e 404).
- **Integração com o front-end**: API REST com CorsFilter, consumida pela aplicação publicada na Vercel.

3. TABELA DE ENDPOINTS (API RESTFUL)

Recurso	Verbo	URI	Descrição	Status esperados
Usuários	GET	/usuarios	Lista todos os usuários	200, 404
Usuários	GET	/usuarios/{codigo}	Busca um usuário pelo código	200, 404
Usuários	POST	/usuarios	Cadastra um novo usuário	201, 400
Usuários	PUT	/usuarios/{codigo}	Atualiza um usuário existente	200, 400, 404
Usuários	DELETE	/usuarios/{codigo}	Remove um usuário	204, 404
Missões	GET	/missoes	Lista todas as missões	200, 404
Missões	GET	/missoes/{codigo}	Busca uma missão pelo código	200, 404

Missões	POST	/missoes	Cadastra uma nova missão	201, 400
Missões	PUT	/missoes/{codigo}	Atualiza uma missão existente	200, 400, 404
Missões	DELETE	/missoes/{codigo}	Remove uma missão	204, 404
Recompensas	GET	/recompensas	Lista todas as recompensas	200, 404
Recompensas	GET	/recompensas/{codigo}	Busca uma recompensa pelo código	200, 404
Recompensas	POST	/recompensas	Cadastra uma nova recompensa	201, 400
Recompensas	PUT	/recompensas/{codigo}	Atualiza uma recompensa existente	200, 400, 404
Recompensas	DELETE	/recompensas/{codigo}	Remove uma recompensa	204, 404
Comunidades/ ONGs	GET	/comunidades	Lista todas as comunidades	200, 404
Comunidades/ ONGs	GET	/comunidades/{codigo}	Busca uma comunidade pelo código	200, 404
Comunidades/ ONGs	POST	/comunidades	Cadastra uma nova comunidade	201, 400
Comunidades/ ONGs	PUT	/comunidades/{codigo}	Atualiza uma comunidade existente	200, 400, 404
Comunidades/ ONGs	DELETE	/comunidades/{codigo}	Remove uma comunidade	204, 404
Usuário-Missão	GET	/usuarios-missoes	Lista as missões dos usuários	200, 404
Usuário-Missão	GET	/usuarios-missoes/{codigo}	Busca um registro pelo	200, 404

			código	
Usuário-Missão	POST	/usuarios-missoes	Registra uma missão para um usuário	201, 400
Usuário-Missão	PUT	/usuarios-missoes/{codigo}	Atualiza o registro (status e data de conclusão)	200, 400, 404
Usuário-Missão	DELETE	/usuarios-missoes/{codigo}	Remove o registro	204, 404
Usuário-Recompensa	GET	/usuarios-recompensas	Lista os resgates	200, 404
Usuário-Recompensa	GET	/usuarios-recompensas/{codigo}	Busca um resgate pelo código	200, 404
Usuário-Recompensa	POST	/usuarios-recompensas	Registra o resgate de uma recompensa	201, 400
Usuário-Recompensa	PUT	/usuarios-recompensas/{codigo}	Atualiza um resgate existente	200, 400, 404
Usuário-Recompensa	DELETE	/usuarios-recompensas/{codigo}	Remove o registro	204, 404

4. PROTÓTIPO DO SISTEMA

O front-end da SoulPet foi desenvolvido em equipe na disciplina de Front-End, com React, Vite e Tailwind CSS. O React estrutura a interface em componentes reutilizáveis, o Vite cuida do ambiente de desenvolvimento e do build, e o Tailwind CSS faz a estilização e deixa as telas responsivas. A aplicação está publicada na Vercel (https://soulpet-fiap.vercel.app) e serve de protótipo da interface da solução, sustentada pela API RESTful descrita neste documento. As telas abaixo foram capturadas do sistema em funcionamento e mostram as principais funcionalidades da solução.

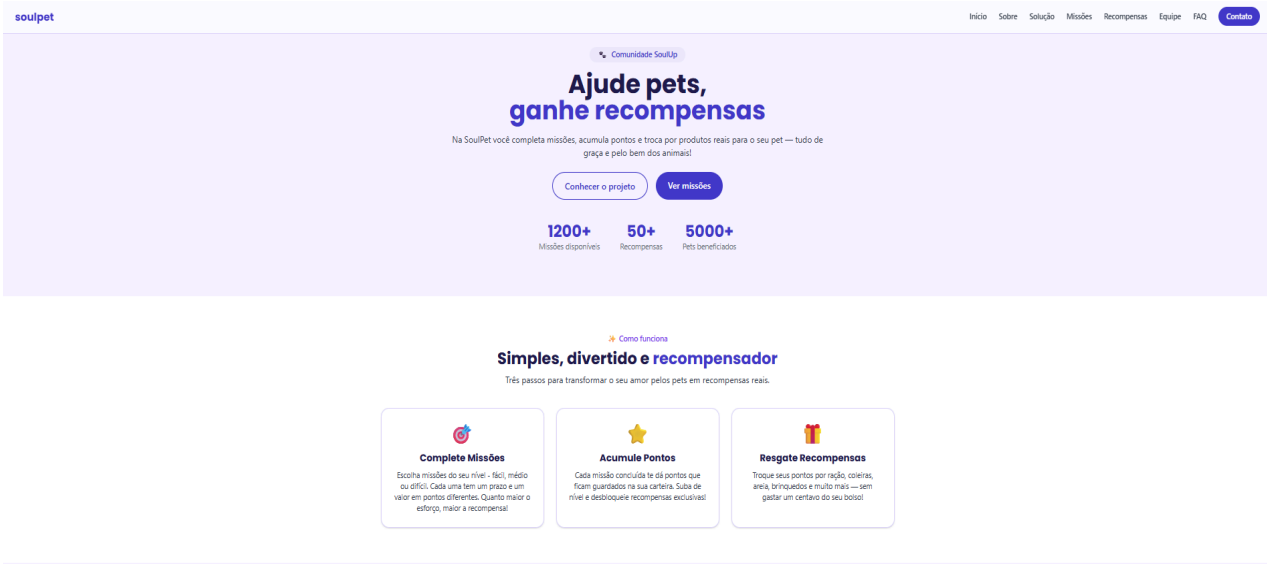


Figura 1: Página Início

Tela de entrada da aplicação. Apresenta a proposta da SoulPet (ajudar pets e ganhar recompensas), traz os botões de acesso ao projeto e às missões e explica em três passos como funciona: completar missões, acumular pontos e resgatar recompensas.



Figura 2: Página Solução

Apresenta o problema que a SoulPet resolve (animais sem cuidado e ONGs dependentes de doações irregulares) e a solução proposta: gamificação com missões, pontos trocados por produtos e doações para ONGs parceiras.

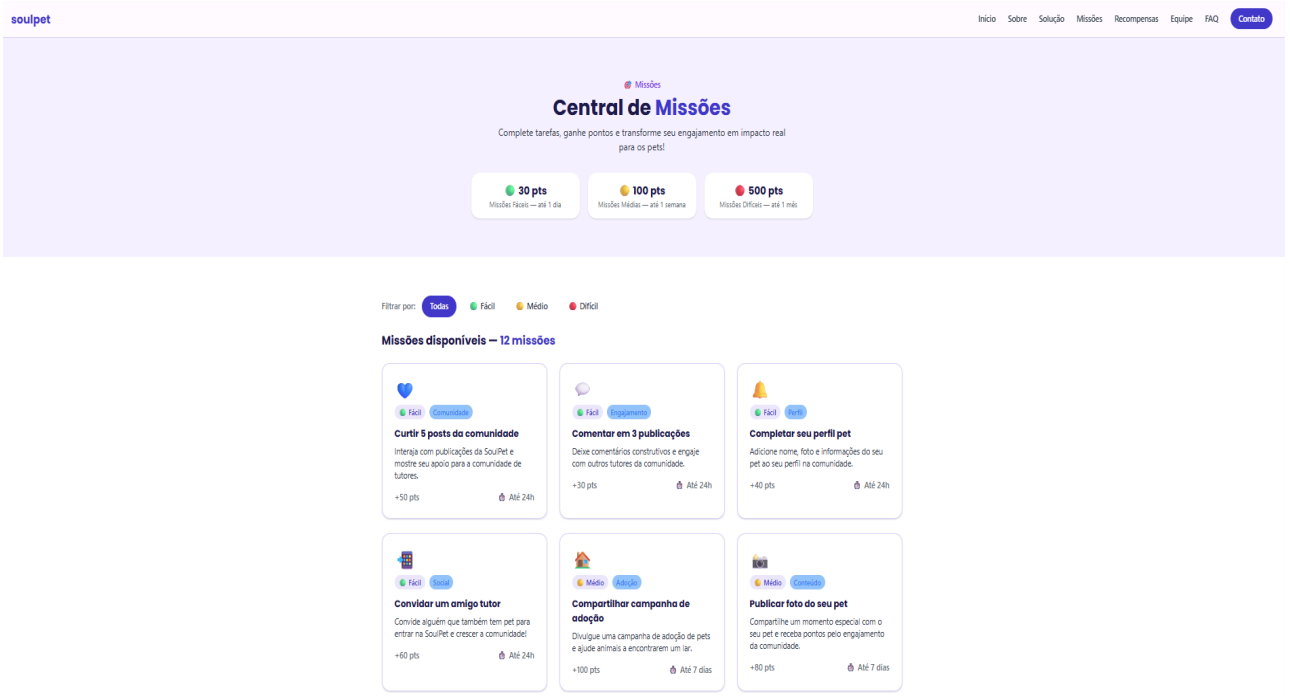


Figura 3: Central de Missões

Lista as missões disponíveis com categoria, descrição, pontuação e prazo, organizadas em cartões.

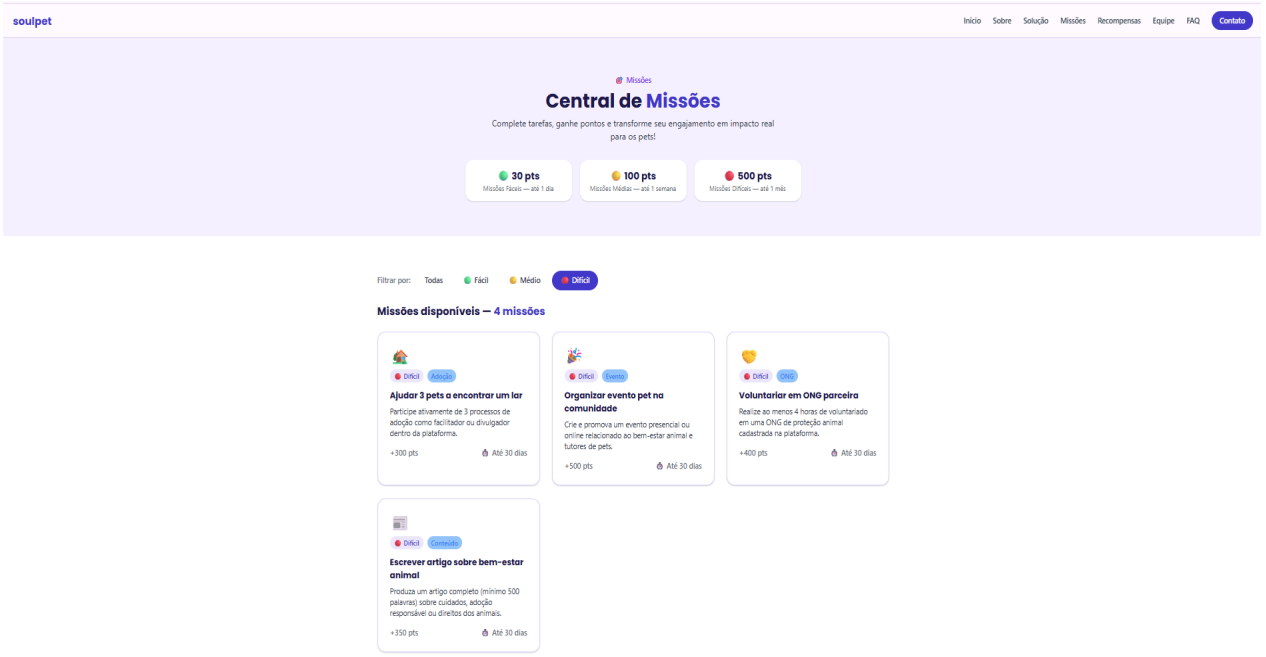


Figura 4: Central de Missões com filtro de dificuldade

Permite filtrar as missões por nível (Fácil, Médio ou Difícil). Na imagem, o filtro Difícil exibe as missões de maior pontuação, com prazo de até 30 dias.

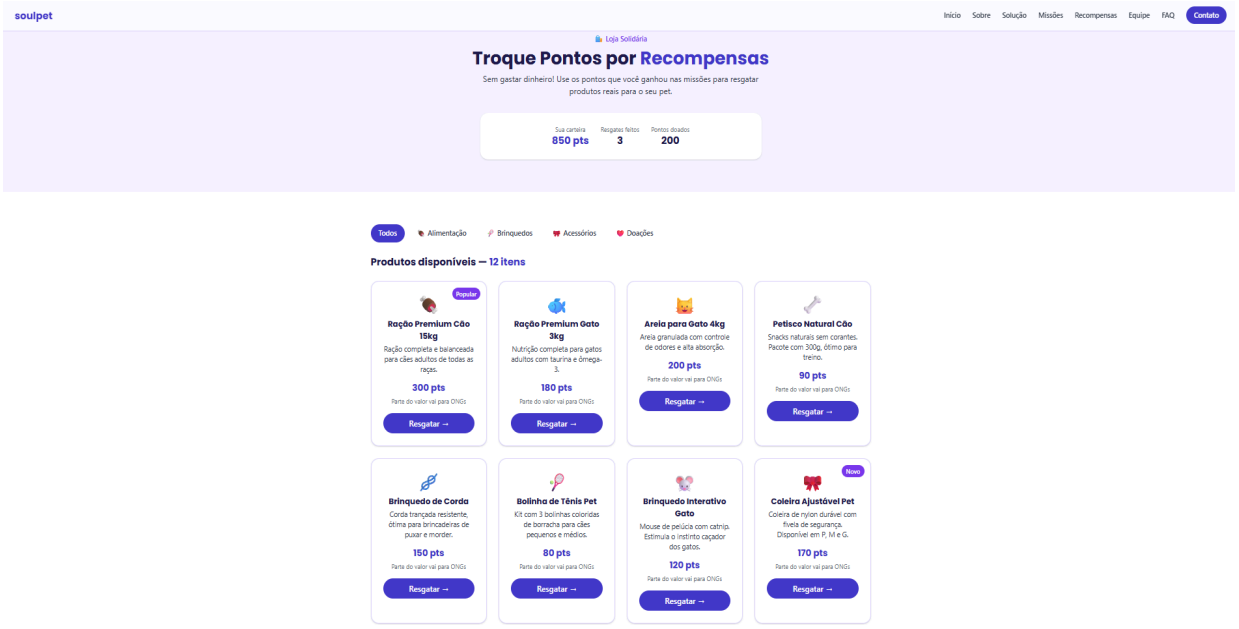


Figura 5: Loja Solidária

Exibe os produtos que podem ser resgatados com pontos, organizados por categoria, e a carteira do usuário.

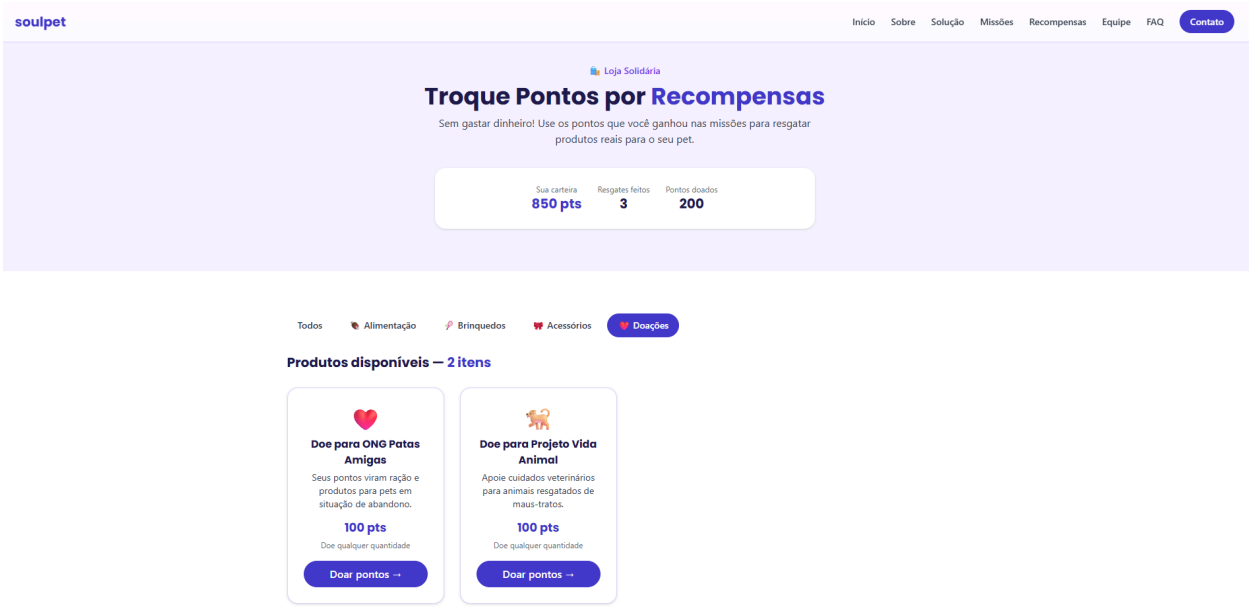


Figura 6: Doações para ONGs parceiras

Mostra a categoria de doações, em que o usuário escolhe doar seus pontos diretamente a ONGs parceiras de proteção animal.

5. MODELO DE ENTIDADE-RELACIONAMENTO (MER)

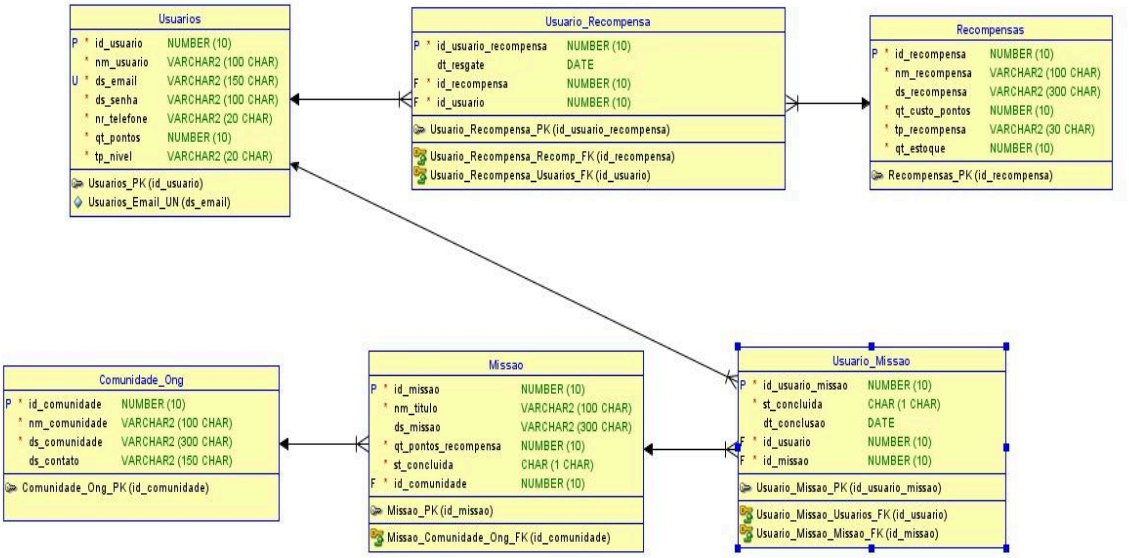


Figura 7: Modelo de Entidade-Relacionamento (MER)

6. DIAGRAMA DE CLASSES

O diagrama de classes representa a organização do projeto Java em camadas: TO (objetos de transferência), DAO (acesso ao banco Oracle, com a ConnectionFactory), BO (regras de negócio) e Resource (endpoints da API). Os atributos são privados e os métodos públicos, o que garante o encapsulamento.

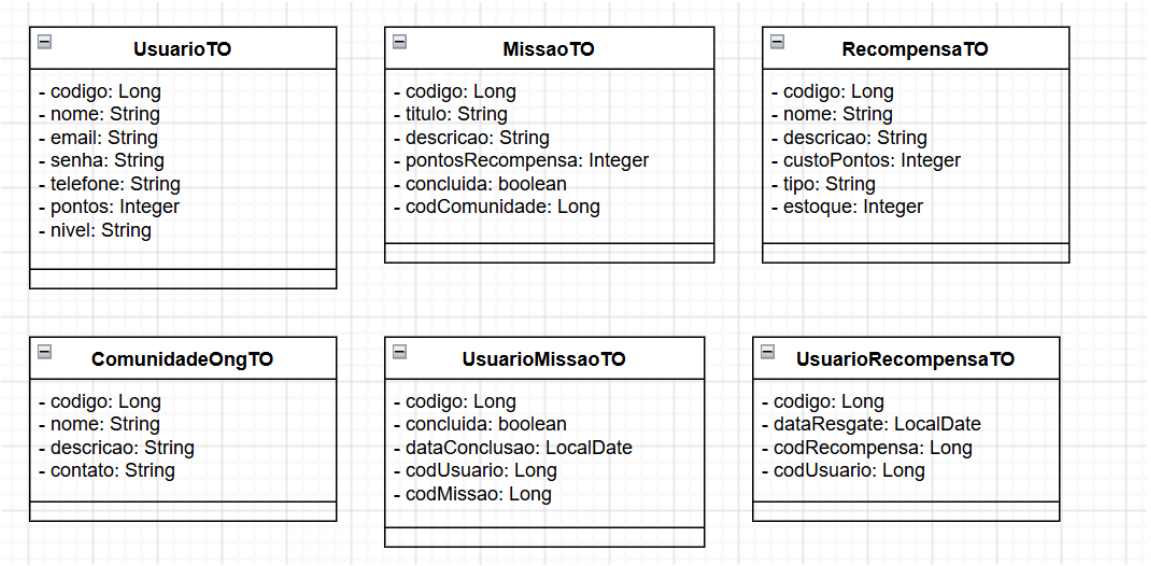


Figura 8: Diagrama de classes - Camada TO

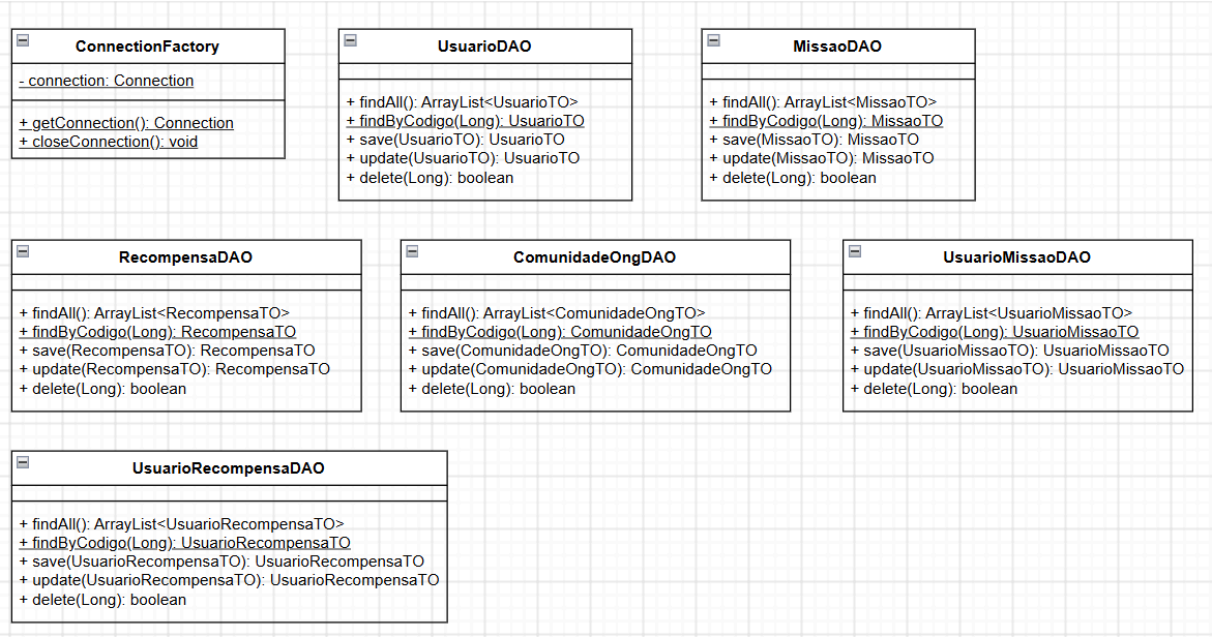


Figura 9: Diagrama de classes - Camada DAO

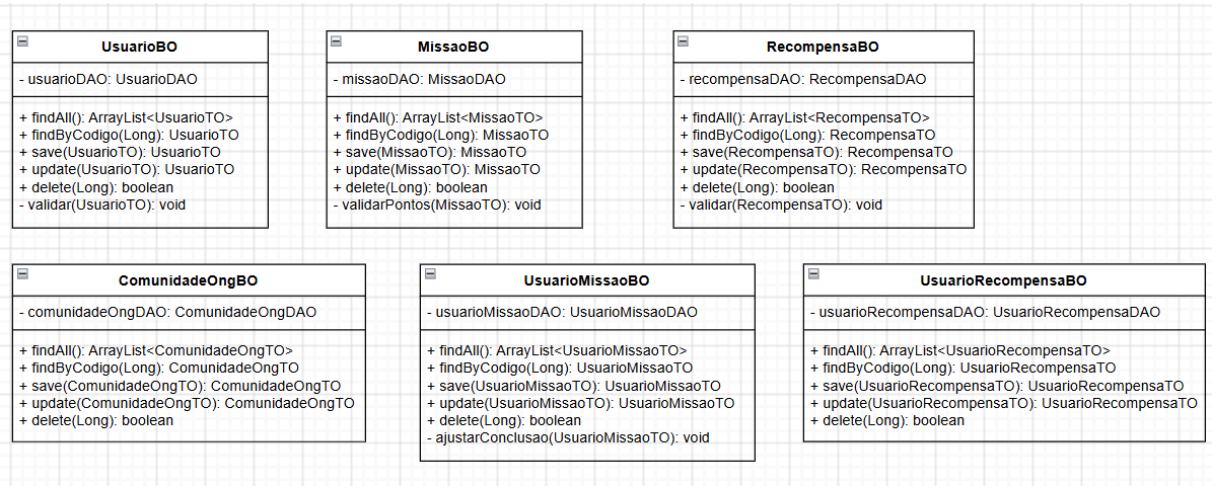


Figura 10: Diagrama de classes - Camada BO

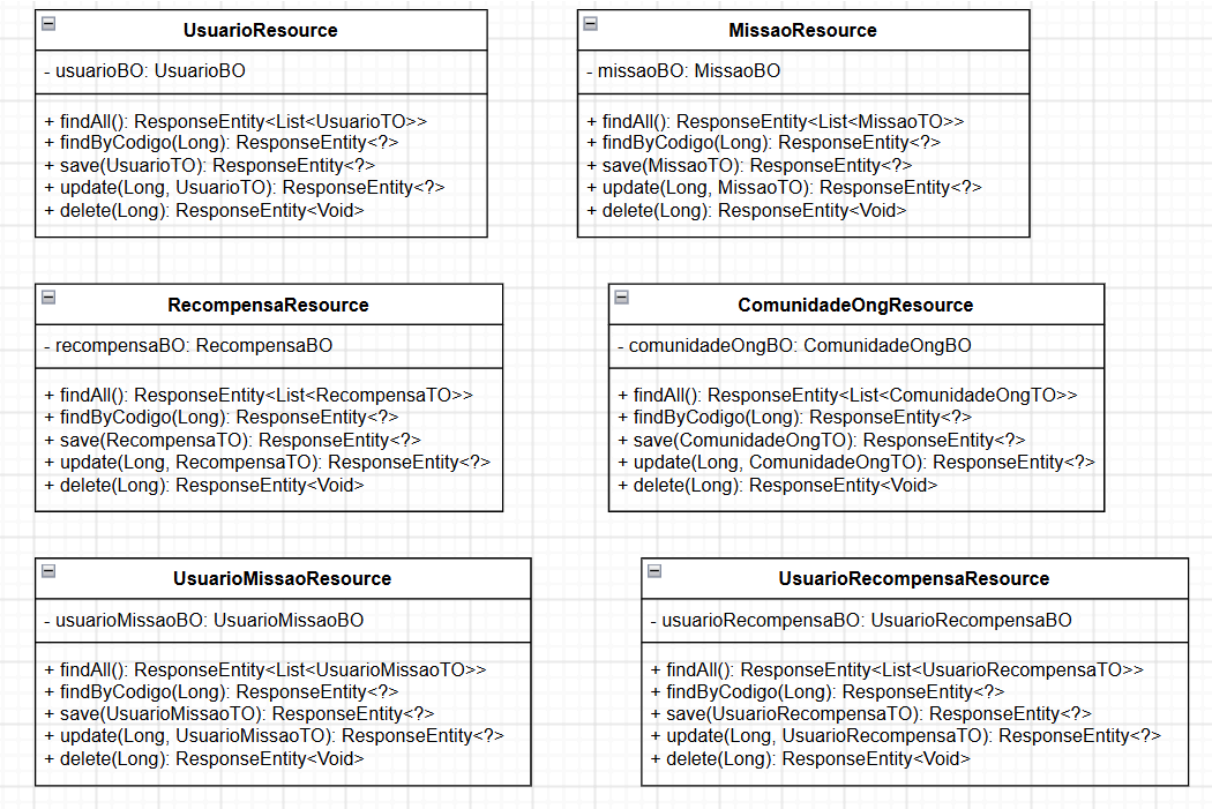


Figura 11: Diagrama de classes - Camada Resource

7. PROCEDIMENTOS PARA RODAR A APLICAÇÃO

7.1 Ferramentas e versões necessárias

- IDE: IntelliJ IDEA
- Java JDK: versão 21
- Gerenciador de dependências: Maven (já incluído no IntelliJ)
- Framework: Spring Boot 4.1.2
- Banco de dados: Oracle, com o driver JDBC ojdbc17 (baixado automaticamente pelo Maven)
- Teste dos endpoints: Insomnia
- Controle de versão: Git

7.2 Como importar o projeto

1. Clone o repositório: git clone https://github.com/biancapereirasilva/soulpet-java-sprint4
2. No IntelliJ, escolha File > Open e selecione a pasta do projeto clonado (a que contém o arquivo pom.xml).

3. Aguarde o IntelliJ importar o projeto Maven e baixar as dependências. Se necessário, clique em Reload All Maven Projects na aba Maven.
4. Em File > Project Structure > Project, confirme que o SDK é o JDK 21.

7.3 Configuração do banco de dados

A conexão com o Oracle é feita pela classe ConnectionFactory (pacote br.com.fiap.model.dao), que lê as credenciais de variáveis de ambiente. Antes de executar, configure as três variáveis abaixo:

- **DB_URL**: URL JDBC do banco Oracle. Exemplo: jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL
- **DB_USER**: usuário de acesso ao banco Oracle.
- **DB_PASSWORD**: senha de acesso ao banco Oracle.

7.4 Executando a aplicação localmente

1. Com as variáveis de ambiente configuradas (item 7.3), abra a classe SoulPetApplication.
2. Clique no botão Run (triângulo verde) ao lado do método main, ou use Shift + F10.
3. Aguarde a mensagem de que a aplicação iniciou no console. Por padrão, o Spring Boot sobe na porta 8080.
4. A API fica disponível em http://localhost:8080.

7.5 Testando os endpoints

1. No Insomnia, crie uma requisição para GET http://localhost:8080/missoes.
2. A resposta esperada é 200 com a lista de missões em JSON.
3. Para os demais verbos (POST, PUT e DELETE), use os caminhos da tabela de endpoints, enviando o corpo em JSON quando necessário.

7.6 Acesso ao deploy

A API também está publicada no Render e pode ser testada sem executar o projeto localmente:

- **URL base da API**: https://soulpet-api.onrender.com
- **Exemplo de requisição**: GET https://soulpet-api.onrender.com/missoes
- **Repositório**: https://github.com/biancapereirasilva/soulpet-java-sprint4

No Render, as mesmas três variáveis de ambiente (DB_URL, DB_USER e DB_PASSWORD) estão configuradas no painel do serviço. Por estar em um plano gratuito, a primeira requisição após um período sem uso pode levar cerca de um minuto para responder.